

Hikmalayer Security Remediation Report

Finding R-05 — Static Admin/P2P Token Secrets (Residual Risk, Low Severity)

Project	Hikmalayer — hikmalayer-core
Organisation	Bestower Labs Limited
Reviewed by	Muhammad Ayan Rao — Founder & Director
Reference document	Hikmalayer Security Assessment & Remediation Plan, 22 June 2026 — Residual Risks and Recommendations, R-05
Report date	16 July 2026
Status	Solution implemented and unit-tested; pending integration into hikmalayer-core and sign-off

1. Finding Summary

The Residual Risks and Recommendations section of the audit records R-05 as a **Low severity** residual risk: **ADMIN_TOKEN** and **P2P_TOKEN** are static, long-lived shared secrets with no expiry and no defined rotation process. The recommendation is twofold: (1) implement a token rotation procedure, and (2) consider time-limited tokens for production.

2. Why This Remediation Method

A static shared secret has one structural weakness regardless of how carefully it is handled: once it exists, it is valid **forever** until someone manually notices a problem and redeploys a new one. It can leak through logs, shell history, a misconfigured CI variable, or an operator leaving the team, and the system has no way to know that has happened. The remediation therefore had to remove that open-ended trust window without introducing new operational overhead the placement-scale project can't sustain.

2.1 Alternatives considered

Option	Why it was not chosen
Manual secret rotation (swap env var, redeploy)	Satisfies the rotation half of the recommendation but not the time-limited half. Requires a redeploy/restart on every rotation, has no built-in expiry, and depends on someone remembering to do it — the exact failure mode R-05 flags.
Database-backed token store with server-side revocation list	Adds a stateful dependency and a lookup on every authorization check, which cuts against the atomic, minimal-surface persistence work already done for HM-05. More moving parts than a Low-severity finding justifies.
Full OAuth2 / external identity provider	Correct for multi-tenant, external-facing systems, but disproportionate for two internal credential types on a small node system — it would add an entire auth-server dependency to close a Low-severity finding.

Option	Why it was not chosen
Off-the-shelf JWT library	Solves the time-limit problem but brings JWT's well-known footguns (algorithm-confusion attacks, "alg: none" bypass, larger dependency surface) for functionality this project only needs a small, auditable slice of.

2.2 Chosen approach: HMAC-signed, self-expiring tokens

The implemented solution mints tokens from a long-lived **signing key** (ADMIN_TOKEN_SIGNING_KEY / P2P_TOKEN_SIGNING_KEY) rather than distributing the credential itself. Each token embeds its own expiry and scope and is HMAC-SHA256 signed, so it verifies itself with no database or session store involved. The reasoning behind each property:

- **Stateless verification** — matches the existing `authorize_admin()` pattern (HeaderMap in, bool out); no new database dependency or added latency per request.
- **Time-limited by construction** — the expiry is inside the signed payload, so it cannot be stripped or extended without invalidating the signature. This directly satisfies the "time-limited tokens" half of the recommendation.
- **Rotation without downtime** — a new token is minted with one CLI command at any time; the app never needs to be redeployed to rotate. Old tokens simply expire on schedule, so multiple P2P peers don't need a synchronised cutover. This directly satisfies the "rotation procedure" half.
- **Fail-closed by default** — if a signing key is unset or malformed, authorization denies rather than silently allowing, consistent with the fail-closed principle already established for HM-04.
- **Constant-time signature comparison** — reuses the timing-side-channel mitigation already applied for HM-07, rather than a plain `==` string check.
- **Scope binding** — an admin-scoped and a p2p-scoped token are cryptographically distinct even if signed with the same key, so a token minted for one purpose cannot be replayed against the other.
- **Emergency revocation still available** — rotating the signing key itself invalidates every outstanding token for that scope immediately, giving both a routine (soft) and an urgent (hard) rotation path.
- **Small, auditable dependency footprint** — only `hmac`, `sha2`, `subtle`, `hex` and `base64` are added, all widely-used, narrowly-scoped crates — proportionate to a Low-severity finding rather than an enterprise auth overhaul.

3. Solution Overview

Token format: a base64 payload of `issued_at:expires_at:scope`, followed by a hex-encoded HMAC-SHA256 signature over that payload.

```
base64(issued_at:expires_at:scope) . hex(hmac_sha256(payload))
```

3.1 Files delivered

File	Purpose
src/auth/token.rs	Token generation and verification: <code>generate_token()</code> , <code>verify_token()</code> , <code>is_valid()</code> , <code>signing_key_from_env()</code> . 8 unit tests.
src/auth/mod.rs	Drop-in <code>authorize_admin()</code> / <code>authorize_p2p()</code> replacements, fail-closed, scope-checked. 5 integration-style tests.
src/bin/mint_token.rs	Operator CLI: mints a fresh signed token for a given scope and TTL. This is the rotation mechanism.
Cargo.toml / src/lib.rs	Crate scaffolding and pinned dependency versions.

3.2 Verification performed

The module was built and tested end-to-end in an isolated environment (Rust 1.75, `cargo build / cargo test`): **13 of 13 tests passed**, covering valid round-trip verification, expiry rejection, scope-mismatch rejection, tampered signature and tampered payload rejection, wrong-signing-key rejection, malformed-token rejection, missing-header denial, and unconfigured-key fail-closed behaviour. The `mint_token` CLI was also run manually: it produced a valid token when a signing key was set, and exited with an error (not a silent pass) when the key was missing.

4. Recommendation Traceability

R-05 recommendation	How it is satisfied
Implement a token rotation procedure	<code>mint_token</code> CLI mints a new token from the signing key at any time, no redeploy required; old tokens expire naturally, removing the need for a synchronised cutover.
Consider time-limited tokens for production	Expiry is embedded in the signed payload and enforced on every verification; a token cannot be used past its <code>expires_at</code> under any circumstance.

5. Integration Steps for hikmalayer-core

- In `state.rs`, replace `admin_token: Option<String>` / `p2p_token: Option<String>` with `admin_token_signing_key: Option<Vec<u8>>` / `p2p_token_signing_key: Option<Vec<u8>>`, populated at startup via `token::signing_key_from_env(...)`.
- In `routes.rs`, replace the existing `authorize_admin` / `authorize_p2p` with the versions in `src/auth/mod.rs`.
- Add `hmac`, `sha2`, `subtle`, `hex`, `base64` to `Cargo.toml` (versions as pinned in the delivered project).

- Update `.env.example` and the deployment runbook: replace `ADMIN_TOKEN=` / `P2P_TOKEN=` with `ADMIN_TOKEN_SIGNING_KEY=` / `P2P_TOKEN_SIGNING_KEY=` (64-character hex, e.g. `openssl rand -hex 32`).

6. Rotation Procedure (Operational)

Routine rotation — mint a new token, distribute it, let the old one expire:

```
ADMIN_TOKEN_SIGNING_KEY=<key> cargo run --bin mint_token -- --scope admin --ttl 86400
P2P_TOKEN_SIGNING_KEY=<key> cargo run --bin mint_token -- --scope p2p --ttl 2592000
```

Emergency revocation — if a token is suspected to have leaked, rotate the signing key itself (generate a new one with `openssl rand -hex 32`, update the env var, restart). This immediately invalidates every outstanding token for that scope, not just the one suspected of leaking.

Recommended TTLs: 8–24 hours for the admin scope (re-minted per operator session or via a scheduled job); 7–30 days for the P2P scope, bounded but long enough for peer continuity.

7. Open Items for Review

- Confirm where the signing keys should live in the production deployment (currently planned as environment variables per the existing `.env.example` convention) — a secrets manager may be preferable given the EC2/Docker Compose deployment already in progress.
- Confirm target TTLs for the admin and P2P scopes for this deployment.
- This closes R-05 in isolation; it does not affect the currently open findings (HM-01, HM-03, HM-06, HM-07), which remain outstanding ahead of production sign-off.